Himanshu Kesarvani
Sr. Data Scientist
HK

Advanced Modeling and Data Science

Advanced Regression & Classification

Swipe to learn more →

Himanshu Kesarvani
Sr. Data Scientist
HK

1. Multicollinearity & VIF

The Concept for a Newbie: Multicollinearity is a problem that occurs in Multiple Linear Regression (MLR) when two or more independent variables (X's) are highly correlated with each other.

- **The Problem:** If X_1 and X_2 are nearly identical, the model can't tell which one is truly driving the change in Y. This makes the individual coefficient estimates (β_1 and β_2) unstable, unreliable, and hard to interpret.
- **Detection:** The **Variance Inflation Factor (VIF)** is the most common way to detect multicollinearity. A VIF value greater than 5 or 10 is usually a cause for concern.

Easy Explanation(The Twin Predictors Analogy): Imagine you are trying to predict a person's weight (Y) using two variables: "Height in Inches" (X_1) and "Height in Centimeters" (X_2). These two variables are almost perfectly correlated (twins!). The model can't decide if it should give all the credit to X_1 or all the credit to X_2, so it gives them wildly unstable coefficients. VIF is a score that tells you how much this "twin problem" is inflating the uncertainty of your coefficients.
Mathematical Foundation: The VIF for a predictor X_i is calculated by regressing X_i on all other predictors:

$$\text{VIF}_i = \frac{1}{1 - R_i^2}$$

Where R_i^2 is the R-squared value from the regression of X_i on the other X's. If X_i is perfectly predicted by the others ($R_i^2 = 1$), VIF goes to infinity.

Himanshu Kesarvani
Sr. Data Scientist
HK

1. Multicollinearity & VIF

Python Code Example (Calculating VIF): We use the `statsmodels` library and a custom function for VIF.

code.py +

```python
from statsmodels.stats.outliers_influence import variance_inflation_factor
import pandas as pd
import numpy as np

# Data with high multicollinearity (X2 is almost a copy of X1)
data = pd.DataFrame({
    'Y': np.random.rand(10),
    'X1': np.arange(10),
    # Highly correlated with X1
    'X2': np.arange(10) + np.random.normal(0, 0.1, 10),
    'X3': np.random.rand(10) # Uncorrelated
})

# Calculate VIF for each predictor
X = data[['X1', 'X2', 'X3']]
vif_data = pd.DataFrame()
vif_data["feature"] = X.columns
vif_data["VIF"] = [variance_inflation_factor(X.values, i) \
                   for i in range(X.shape[1])]
print(f"\n--- Variance Inflation Factor (VIF) ---")
print(vif_data)
# X1 and X2 will have very high VIFs, indicating multicollinearity.
```

>>> Output

```
--- Variance Inflation Factor (VIF) ---
  feature          VIF
0      X1  2560.399540
1      X2  2605.944978
2      X3     2.269037
```

Follow for more!

Himanshu Kesarvani
Sr. Data Scientist
HK

2. Logistic Regression (Binary)

The Concept for a Newbie: Logistic Regression is a statistical model used for **classification** problems, where the dependent variable (Y) is **categorical** (usually binary, e.g., Yes/No, 0/1, Buy/Not Buy).

- **Goal:** To predict the *probability* that an observation belongs to one of the categories.
- **Key Difference from Linear Regression:** It does not predict Y directly. Instead, it uses a special function (the **Logit Link**, Topic 58) to transform the linear combination of X's into a probability (a value between 0 and 1).

Easy Explanation(The Probability S-Curve Analogy): Imagine you want to predict if a student will pass (1) or fail (0) a test based on hours studied (X). Linear regression would try to draw a straight line, which is bad because a score can't be less than 0 or greater than 1. Logistic Regression uses an S-shaped curve (the sigmoid function) that smoothly goes from 0 to 1. This curve represents the probability of passing.

Mathematical Foundation: The core of Logistic Regression is the Sigmoid (or Logistic) function, which maps any real number to a value between 0 and 1:

$$P(Y = 1 \mid X) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 X)}}$$

Himanshu Kesarvani
Sr. Data Scientist
HK

2. Logistic Regression (Binary)

Python Code Example (Logistic Regression): We use `statsmodels` for the statistical approach.

code.py +

```python
import pandas as pd
import statsmodels.api as sm
import numpy as np

# Data: Hours studied (X) and Pass/Fail (Y)
data = pd.DataFrame({
    'Hours': [0.5, 1, 2, 3, 4, 5, 6, 7, 8, 9],
    # 0=Fail, 1=Pass - Modified to remove perfect separation
    'Pass': [0, 0, 1, 0, 1, 1, 1, 1, 1, 1]
})

X = sm.add_constant(data['Hours']) # Add intercept term
Y = data['Pass']

# Define and fit the Logistic Regression model
logit_model = sm.Logit(Y, X)
results = logit_model.fit(disp=0) # disp=0 suppresses output during fitting

print(f"\n--- Logistic Regression ---")
print(f"Intercept (beta_0): {results.params['const']:.2f}")
print(f"Hours (beta_1): {results.params['Hours']:.2f}")
# The coefficients are interpreted in terms of log-odds (Topic 58).
```

>>> Output

```
--- Logistic Regression ---
Intercept (beta_0): -3.28
Hours (beta_1): 1.30
```

Follow for more!

Himanshu Kesarvani
Sr. Data Scientist
HK

3. Odds Ratio and Logit Link

The Concept for a Newbie: To interpret the coefficients (β) in Logistic Regression, we need to understand **Odds** and the **Logit Link**.

- **Odds:** The ratio of the probability of an event happening to the probability of it not happening. $\text{Odds} = P/(1-P)$.
- **Logit Link:** The natural logarithm of the odds. $\text{Logit}(P) = \ln(\text{Odds})$. This is the linear part of the model: $\text{Logit}(P) = \beta_0 + \beta_1 X$.
- **Odds Ratio (OR):** When you exponentiate the coefficient (e^{β_1}), you get the Odds Ratio. The OR tells you how the odds of the outcome change for a one-unit increase in X.

Easy Explanation(The Bet Analogy): Imagine betting on a horse.

- **Odds of 3:1** means for every 1 time the horse loses, it wins 3 times. $P(\text{Win}) = 3/4 = 0.75$.
- The Logistic Regression coefficient (β_1) is like a "log-odds multiplier." If $e^{\beta_1} = 2$, it means that for every extra hour studied, the odds of passing the test **double**.

Mathematical Foundation:

$$\text{Odds Ratio} = e^{\beta_1}$$

If $\text{OR} > 1$, the odds of the event increase with X. If $\text{OR} < 1$, the odds decrease. If $\text{OR} = 1$, X has no effect.

Himanshu Kesarvani
Sr. Data Scientist
HK

3. Odds Ratio and Logit Link

Python Code Example (Calculating Odds Ratio): We exponentiate the coefficients from the Logistic Regression results.

code.py +

```
# Using the results from Topic 57
odds_ratio_hours = np.exp(results.params['Hours'])

print(f"\n--- Odds Ratio and Logit Link ---")
print(f"Odds Ratio for Hours: {odds_ratio_hours:.2f}")
print(f"Interpretation: For every 1-unit increase in Hours Studied, \
the odds of Passing the test increase by a factor of {odds_ratio_hours:.2f}.")
```

>>> Output

```
--- Odds Ratio and Logit Link ---
Odds Ratio for Hours: 3.68
Interpretation: For every 1-unit increase in Hours Studied, the odds of Passing the test increase by a factor of
3.68.
```

Himanshu Kesarvani
Sr. Data Scientist
HK

4. Confusion Matrix & Accuracy

The Concept for a Newbie: When evaluating a classification model (like Logistic Regression), we use a **Confusion Matrix** to summarize the performance. It compares the model's predictions to the actual outcomes.

	Predicted Positive	Predicted Negative
Actual Positive	True Positive (TP)	False Negative (FN)
Actual Negative	False Positive (FP)	True Negative (TN)

- **Accuracy:** The most intuitive metric. It is the proportion of total predictions that were correct.

$$\text{Accuracy} = \frac{TP + TN}{\text{Total}}$$

Easy Explanation(The Doctor's Diagnosis Analogy): Imagine a doctor diagnosing a disease (Positive) or not (Negative).

- **TP:** The doctor correctly says you have the disease.
- **TN:** The doctor correctly says you don't have the disease.
- **FP (Type I Error):** The doctor says you have the disease, but you don't (False Alarm).
- **FN (Type II Error):** The doctor says you don't have the disease, but you do (Missed Diagnosis).

Accuracy is the percentage of all patients the doctor got right.

Himanshu Kesarvani
Sr. Data Scientist
HK

4. Confusion Matrix & Accuracy

Mathematical Foundation: While Accuracy is simple, it can be misleading, especially with imbalanced data (e.g., 99% of people don't have the disease, so a model that always predicts "No Disease" would have 99% accuracy but be useless). This is why we need other metrics

code.py +

```python
from sklearn.metrics import confusion_matrix, accuracy_score
import numpy as np

# Actual outcomes (Y_true) and Model predictions (Y_pred)
Y_true = np.array([1, 0, 1, 1, 0, 1, 0, 1, 0, 1])
Y_pred = np.array([1, 0, 1, 0, 0, 1, 1, 1, 0, 1])

# Calculate the Confusion Matrix
cm = confusion_matrix(Y_true, Y_pred)

# Calculate Accuracy
accuracy = accuracy_score(Y_true, Y_pred)

print(f"\n--- Confusion Matrix and Accuracy ---")
print(f"Confusion Matrix:\n{cm}")
print(f"Accuracy: {accuracy:.2f}")
# In this case, 8 out of 10 predictions were correct (80% accuracy).
```

>>> Output

```
--- Confusion Matrix and Accuracy ---
Confusion Matrix:
[[3 1]
 [1 5]]
Accuracy: 0.80
```

Himanshu Kesarvani
Sr. Data Scientist
HK

5. Precision, Recall, and F1-Score

The Concept for a Newbie: These metrics provide a more nuanced view of classification performance than simple Accuracy, especially for imbalanced datasets.

- **Precision (Positive Predictive Value):** Out of all the times the model predicted "Positive," how many were actually correct? (Focuses on avoiding False Positives).

$$\text{Precision} = \frac{TP}{TP + FP}$$

- **Recall (Sensitivity):** Out of all the actual "Positive" cases, how many did the model correctly identify? (Focuses on avoiding False Negatives).

$$\text{Recall} = \frac{TP}{TP + FN}$$

- **F1-Score:** The harmonic mean of Precision and Recall. It provides a single score that balances both metrics. It is often the preferred metric for overall model performance.

$$\text{F1-Score} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

Easy Explanation(The Fishing Net Analogy Revisited): You are fishing for a rare type of fish (Positive).

- **Precision:** Out of all the fish you caught (Predicted Positive), what fraction were the rare fish? (How precise was your net?)
- **Recall:** Out of all the rare fish in the ocean (Actual Positive), what fraction did you manage to catch? (How many did you recall from the ocean?)
- **F1-Score:** A single score that tells you if your net is both precise (not catching junk) and recalling (not missing the rare fish).

Himanshu Kesarvani
Sr. Data Scientist
HK

5. Precision, Recall, and F1-Score

Mathematical Foundation: The choice of metric depends on the business problem. For a spam filter, high **Precision** is key (you don't want to falsely flag a good email). For a disease diagnosis, high **Recall** is key (you don't want to miss a sick person).
Python Code Example (Calculating P, R, F1): We use `sklearn.metrics.classification_report`.

code.py

```
from sklearn.metrics import classification_report
import numpy as np

Y_true = np.array([1, 0, 1, 1, 0, 1, 0, 1, 0, 1])
Y_pred = np.array([1, 0, 1, 0, 0, 1, 1, 1, 0, 1])

report = classification_report(Y_true, Y_pred, \
                               target_names=['Class 0', 'Class 1'], \
                               output_dict=True)

print(f"\n--- Precision, Recall, and F1-Score ---")
print(f"Precision (Class 1): {report['Class 1']['precision']:.2f}")
print(f"Recall (Class 1): {report['Class 1']['recall']:.2f}")
print(f"F1-Score (Class 1): {report['Class 1']['f1-score']:.2f}")
```

>>> Output

```
--- Precision, Recall, and F1-Score ---
Precision (Class 1): 0.83
Recall (Class 1): 0.83
F1-Score (Class 1): 0.83
```